Die Langfassung · 12 Teile in 4 Stufen · Mac und Windows

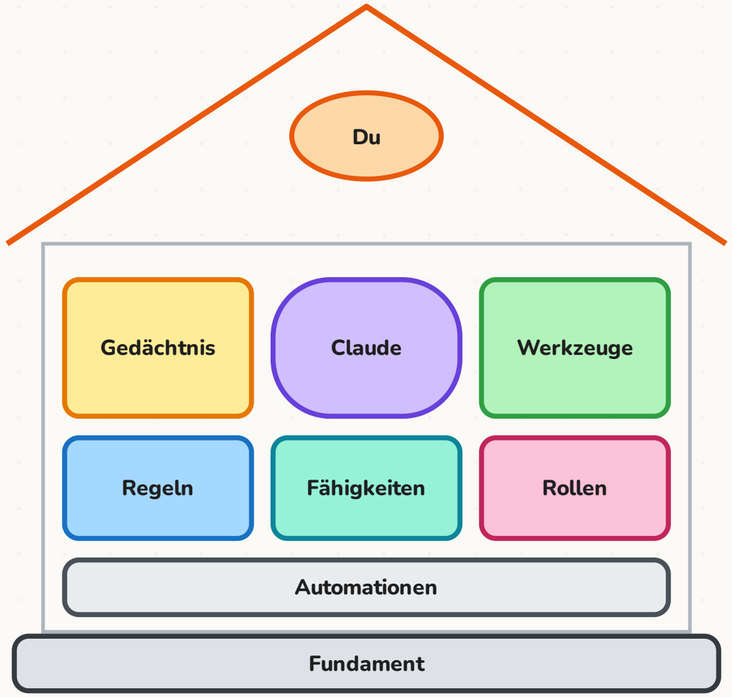
Vom Chat zum KI-Betriebssystem

Schritt für Schritt vom ersten Gespräch mit Claude Code bis zu einem System, das sich erinnert, Regeln kennt, Werkzeuge benutzt, sich selbst prüft und aus Fehlern lernt.

rund 8 Stunden zum Mitmachen, verteilt

jeder Teil für sich nützlich

Stand 20.10.2026



Die 12 Teile auf einen Blick

Stufe A: Loslegen

- **01** Claude Code installieren
  ca. 30 Min. zum Mitmachen
- **02** Der Arbeitsplatz
  ca. 45 Min. zum Mitmachen
- **03** Die Hausordnung
  ca. 30 Min. zum Mitmachen
- **04** Das Kurzzeitgedächtnis
  ca. 30 Min. zum Mitmachen

Stufe B: Absichern und erweitern

- **05** Sichern
  ca. 60 Min. zum Mitmachen
- **06** Werkzeuge anbinden
  ca. 45 Min. zum Mitmachen
- **07** Rezepte und Rollen
  ca. 45 Min. zum Mitmachen

Stufe C: Verlässlich bauen

- **08** Projekte planen
  ca. 30 Min. Mitmachen je Vorhaben
- **09** Prüfen statt glauben
  ca. 45 Min. zum Mitmachen

Stufe D: Das System lernt und läuft allein

- **10** Die Knowledge Base
  ca. 45 Min. zum Mitmachen
- **11** Automationen
  ca. 60 Min. zum Mitmachen
- **12** Mehrere Rechner und Dauerbetrieb
  nach Bedarf

Die Zeit je Teil ist die Zeit zum Mitmachen am eigenen Rechner. Jeder Teil endet mit „Fertig, wenn …“: einem Check, den du selbst machen kannst.

Teil 01 von 12 · Stufe A: Loslegen

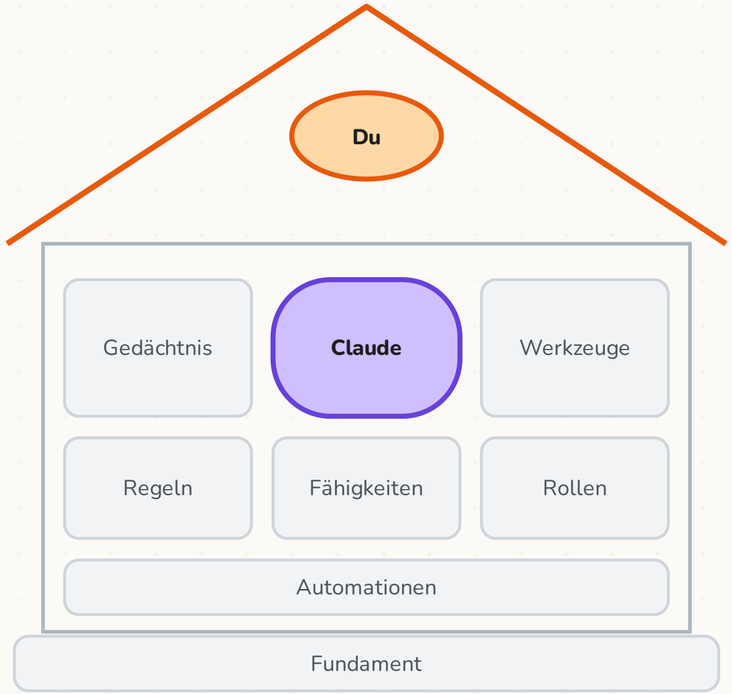
Claude Code installieren

Claude Code läuft auf deinem Rechner und hat seine erste echte Aufgabe in einem Ordner erledigt.

ca. 30 Min. zum Mitmachen

keine Vorkenntnisse

Claude-Abo ab Pro



Vom Chatfenster zum Agenten

Das Problem

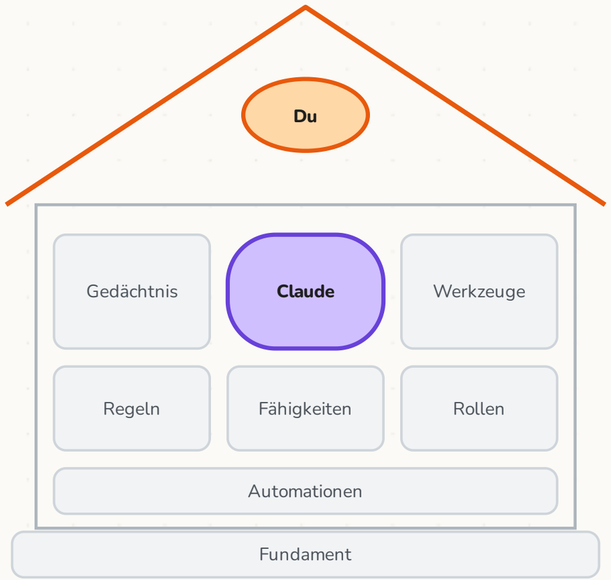
Ein Chat sieht deine Dateien nicht. Alles muss hinein- und wieder herauskopiert werden, und nach dem Gespräch ist es weg.

Das Prinzip

Claude Code ist ein Agent. Er arbeitet in einem Ordner auf deinem Rechner, liest und schreibt Dateien und führt Befehle aus. Jede Aktion wird vorher geprüft: Harmloses erledigt er selbst, Riskantes wird blockiert.

So mache ich es

Ich starte Claude in einem Ordner. Alles, was Claude tut, landet dort als Datei, die ich sehen, ändern und löschen kann.



Was du brauchst

Claude-Abo

Zugang zu Claude Code. Im kostenlosen Plan ist Claude Code nicht enthalten.

Kosten: Pro 18 €/Monat, 15 €/Monat bei Jahreszahlung; Max ab 90 €/Monat

Offizielle Seite ↗

Claude Code

Der KI-Agent, der in deinem Ordner liest, schreibt und Befehle ausführt. Läuft auf Mac (ab macOS 13) und Windows (ab 10, Version 1809) ohne Zusatzsoftware.

Kosten: im Claude-Abo enthalten (ab Pro)

Offizielle Seite ↗ GitHub ↗

Claude Desktop (Code-Tab)

Dieselbe Arbeitsweise mit Fenster statt Terminal. Gut für den allerersten Versuch.

Kosten: wie Claude Code

Offizielle Seite ↗ Anleitung ↗

Ohne Terminal geht es auch: der Code-Tab in Claude Desktop. Die Anleitung nutzt das Terminal, weil dort alles genauso funktioniert und nichts versteckt ist.

Installieren in vier Schritten

1 Terminal öffnen

Mac: `Cmd + Leertaste`, „Terminal“ tippen, Enter. Windows: Startmenü, „PowerShell“ tippen, Enter.

2 Claude Code installieren

Befehl kopieren, ins Terminal einfügen, Enter.

Mac · Terminal

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows · PowerShell

```
irm https://claude.ai/install.ps1 | iex
```

3 Neues Terminal-Fenster öffnen und prüfen

Erscheint eine Versionsnummer, ist alles installiert.

```
claude --version
```

4 Starten und anmelden

`claude` eingeben. Der Browser öffnet sich, dort mit deinem Claude-Konto anmelden.

Fünf Befehle reichen für den Anfang

Im Terminal

- `pwd` zeigt, in welchem Ordner du gerade bist
- `ls` listet, was in diesem Ordner liegt
- `mkdir wochenbericht` legt einen Ordner an
- `cd wochenbericht` wechselt hinein, `cd ..` eine Ebene zurück
- `claude` startet Claude in genau diesem Ordner

In Claude

- Normal schreiben, wie in einem Chat
- `Esc` stoppt Claude mitten in der Arbeit
- `/help` zeigt alle eingebauten Befehle
- `Shift + Tab` wechselt den Modus. Bei „manual mode“ fragt Claude vor jeder Änderung
- `/exit` beendet die Sitzung

Alle fünf Terminal-Befehle funktionieren auf dem Mac und in der Windows-PowerShell gleich.

Die erste echte Aufgabe

- → Einen Ordner anlegen und Claude darin starten (Befehle rechts).
- → Eine Aufgabe geben, die eine Datei erzeugt. Claude legt sie direkt an, eine neue Datei gilt als harmlos. Im „manual mode“ fragt es vorher.
- → Die Datei im Finder oder Explorer öffnen: Sie liegt wirklich auf deinem Rechner.
- → Nachfragen, korrigieren lassen, weiterbauen. So lernt man Claude kennen.

Im Terminal, Zeile für Zeile

```
mkdir wochenbericht
cd wochenbericht
claude

> Leg eine Datei woche-43 an und notier darin,
  was ich diese Woche erledigt habe:
  Präsentation fürs Team fertig, zwei Angebote
  verschickt, Urlaub eingetragen.
```

Sag Claude

Ab jetzt kannst du Claude alles fragen

> Ich bin neu bei Claude Code und habe noch nie mit dem Terminal gearbeitet. Erklär mir in einfachen Worten, was du in diesem Ordner tun kannst, was ein Terminal ist und wie ich dich sicher bediene. Frag mich danach, wofür ich dich nutzen möchte, und schlag mir drei erste Aufgaben vor.

Was passiert Claude erklärt sich selbst und schlägt Aufgaben vor, die zu dir passen.

Warum Jeder weitere Teil dieser Anleitung lässt sich so umsetzen: Prompt kopieren, Claude macht es und erklärt dabei.

Fertig, wenn …

- [ ] `claude --version` zeigt eine Versionsnummer
- [ ] Claude hat in deinem Ordner eine Datei angelegt, die du im Finder oder Explorer siehst
- [ ] du weißt, wie du Claude stoppst (`Esc`) und beendest (`/exit`)

Stolpersteine

- ! „Befehl nicht gefunden“: ein neues Terminal-Fenster öffnen, das alte kennt den Befehl noch nicht
- ! Mit dem kostenlosen Claude-Plan klappt die Anmeldung nicht, Claude Code braucht mindestens Pro
- ! Auf verwalteten Firmenrechnern kann die Installation gesperrt sein

Weiter mit Teil 02: **Der Arbeitsplatz**

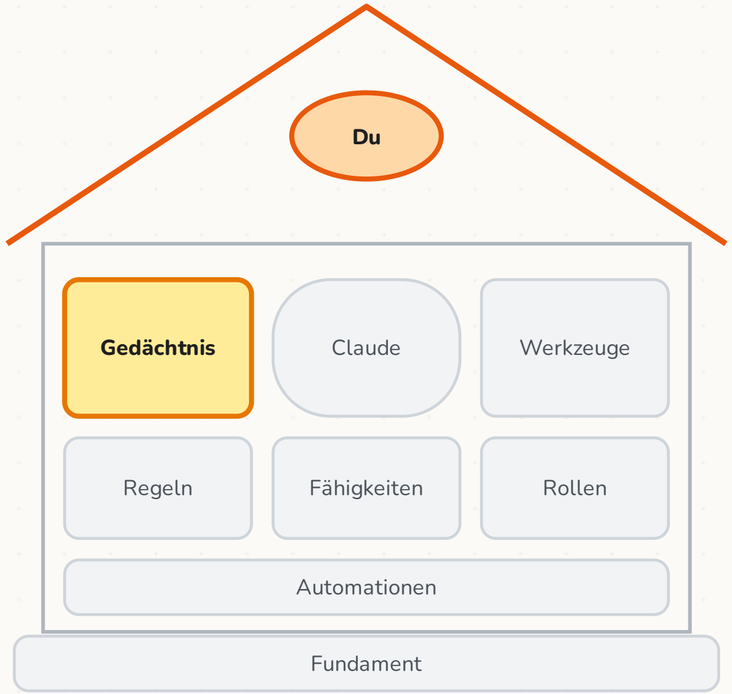
Zur Übersicht aller Teile

Teil 02 von 12 · Stufe A: Loslegen

Der Arbeitsplatz

Deine Notizen liegen in einem Obsidian-Ordner, und du sprichst mit Claude direkt neben ihnen.

ca. 45 Min. zum Mitmachen | Voraussetzung: Teil 1 | kostenlos



Notizen, die Claude lesen kann

Das Problem

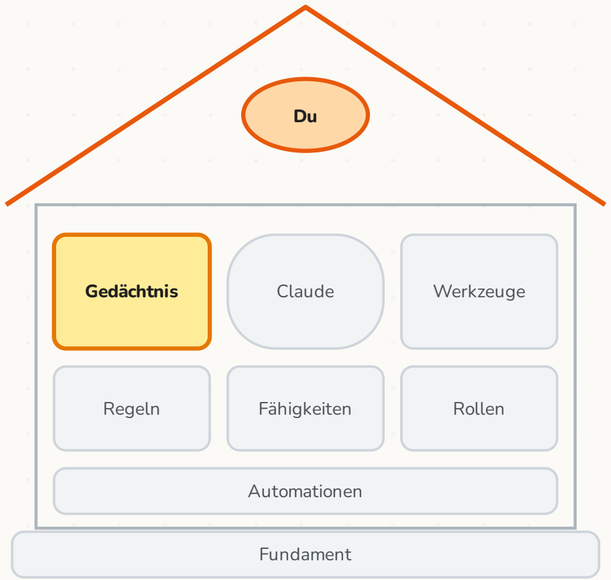
Das Terminal ist ungewohnt, und deine Notizen liegen verstreut in Apps, an die Claude nicht herankommt.

Das Prinzip

Notizen sind einfache Textdateien in einem Ordner, dem Vault. Obsidian zeigt sie schön an, Claude liest und schreibt dieselben Dateien wie du.

So mache ich es

Mein Vault ist mein zweites Gehirn. Claude läuft als Plugin direkt in Obsidian. Für größere Software-Projekte nehme ich VS Code mit Claude im Terminal.



Was du brauchst

Obsidian

Notiz-App, deren Notizen normale Textdateien in einem Ordner sind. Genau diese Dateien liest und schreibt Claude.

Kosten: kostenlos, auch beruflich; Sync und Publish optional

Offizielle Seite ↗ Anleitung ↗

Claudian (Obsidian-Plugin)

Chat mit Claude Code direkt in Obsidian. Im Katalog: Autor **Yishen Tu**, ID `realclaudian`. Es gibt Nachbauten mit ähnlichem Namen.

Kosten: kostenlos (MIT-Lizenz); Claude-Abo nötig

Offizielle Seite ↗ GitHub ↗

VS Code

Code-Editor für größere Software-Projekte. Claude Code läuft darin im Terminal oder als Erweiterung.

Kosten: kostenlos

Offizielle Seite ↗ GitHub ↗

VS Code ist optional und erst ab Teil 8 interessant.

Obsidian und Claudian einrichten

1 Obsidian installieren und den Ordner öffnen

Beim ersten Start „Ordner als Vault öffnen“ und den Ordner `wochenbericht` aus Teil 1 wählen. Neu anfangen geht mit „Einen neuen Vault erstellen“.

2 Externe Erweiterungen zulassen

Einstellungen (Zahnrad unten links) › „Externe Erweiterungen“ › „Eingeschränkten Modus verlassen“.

3 Claudian installieren

„Durchsuchen“, nach **Claudian** suchen, Autor **Yishen Tu** (mit Abstand die meisten Downloads), Installieren, Aktivieren.

4 Ein Modell freischalten

Einstellungen › Claudian › Providers › Models: Haken bei **Sonnet**. Danach unten im Chat das Modell wählen.

5 Erste Nachricht

Claudian-Symbol in der linken Leiste öffnen und schreiben: „Fass mir woche-43 in einem Satz zusammen.“ Unter Windows Obsidian vorher einmal neu starten.

Eine Ordnerstruktur, die mitwächst

Für den Start

- `00 Kontext`: wer du bist, wie du schreibst, was du anbietest
- `01 Inbox`: alles, was noch keinen Platz hat
- `02 Projekte`: eine Datei je Vorhaben
- `06 Tagesnotizen`: eine Notiz je Tag (Teil 4)
- `07 Archiv`: Abgeschlossenes, nichts wird gelöscht

Warum so

- Nummern halten die Reihenfolge fest, Lücken lassen Platz für später
- `05 Knowledge Base` kommt in Teil 10 dazu
- Claude findet sich zurecht, weil jede Datei einen festen Ort hat
- Dateinamen dürfen Leerzeichen und Umlaute haben

Sag Claude

Lass Claude den Vault einrichten

> Leg in diesem Vault die Ordner „00 Kontext“, „01 Inbox“, „02 Projekte“, „06 Tagesnotizen“ und „07 Archiv“ an. Dann interviewe mich mit fünf Fragen (Beruf, typische Aufgaben, Werkzeuge, Ziele, wie ich Texte mag) und schreib daraus die Datei „00 Kontext/Über mich.md“.

Was passiert Claude legt die Ordner an, stellt die Fragen nacheinander und schreibt dein Profil.

Warum Ab jetzt kann Claude in jeder Sitzung nachlesen, wer du bist und wie du arbeitest.

Fertig, wenn …

- [ ] Claude antwortet dir in Obsidian im Claudian-Fenster
- [ ] die fünf Ordner sind im Vault sichtbar
- [ ] „00 Kontext/Über mich.md“ beschreibt dich so, dass du zustimmst

Stolpersteine

- ! Plugins mit ähnlichem Namen (Claudian Plus, Oh My Claudian …): auf den Autor Yishen Tu achten
- ! Claudian findet Claude nicht: Claude Code muss installiert sein, danach Obsidian neu starten
- ! „Model unavailable“ im Chat: in Claudian ist noch kein Modell freigeschaltet (Providers › Models)
- ! Erweiterungen führen fremden Code aus; der Katalog prüft sie automatisch, aber nicht von Hand

Weiter mit Teil 03: **Die Hausordnung**

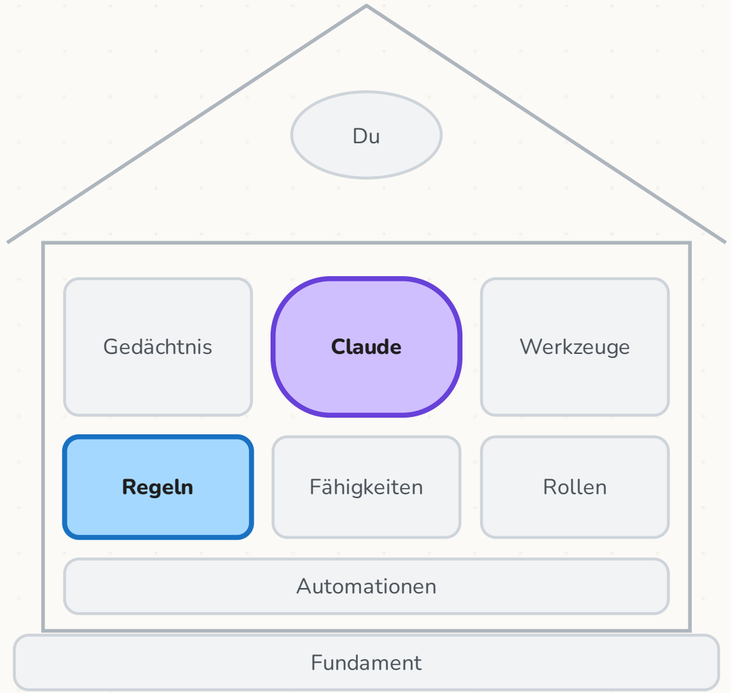
Zur Übersicht aller Teile

Teil 03 von 12 · Stufe A: Loslegen

Die Hausordnung

Claude kennt deine Regeln bei jedem Start, ohne dass du sie wiederholst.

ca. 30 Min. zum Mitmachen | Voraussetzung: Teil 2 | kostenlos



Regeln, die Claude bei jedem Start liest

Das Problem

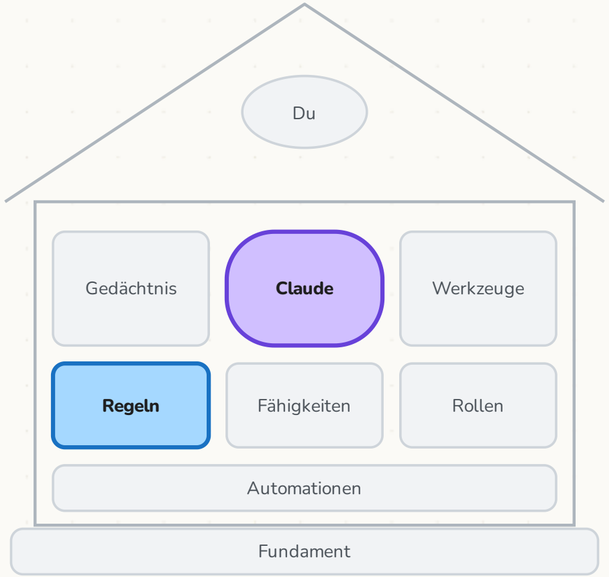
Ohne Regeln beginnt jede Sitzung bei null: Sprache, Ton, Vorlieben und No-Gos erklärst du jedes Mal neu.

Das Prinzip

Eine Textdatei namens CLAUDE.md. Claude Code lädt sie bei jedem Start automatisch. Was darin steht, gilt, bis du es änderst.

So mache ich es

Eine globale Datei für alles, rund 80 Zeilen. Dazu je Projekt eine eigene für das Besondere. Früher waren es über 200 Zeilen; gekürzt wurde, was nie wirkte.



Zwei Ebenen

Global: gilt überall

- Mac: `~/.claude/CLAUDE.md`
- Windows: `%USERPROFILE%\.claude\CLAUDE.md`
- Sprache, Ton, Arbeitsweise
- was Claude nie ohne Rückfrage tut
- wie „fertig“ belegt wird

Projekt: gilt nur dort

- `CLAUDE.md` im Projektordner
- Zweck und Stand des Projekts
- Technik, Ordner, wichtige Dateien
- bekannte Fallen
- `/init` erzeugt einen ersten Entwurf

Prüfen: Frag „Welche CLAUDE.md-Dateien hast du gerade geladen?“. Im Terminal zeigt `/memory` dasselbe und öffnet die Dateien zum Bearbeiten.

Ein Anfang in 15 Zeilen

- → Kurz statt vollständig: jede Regel kostet bei jedem Start Aufmerksamkeit.
- → Verhalten beschreiben, das man prüfen kann („erst Plan zeigen“), keine Wünsche („sei gründlich“).
- → Wenn du Claude korrigierst: „Mach daraus eine Regel.“ So wächst die Datei aus echten Fehlern.
- → Verweisen statt kopieren: Dein Profil liegt in `00 Kontext`, die Regel zeigt nur dorthin.

~/.claude/CLAUDE.md

```
# Meine Hausordnung

## Sprache
- Antworte auf Deutsch, kurz und direkt.

## Arbeitsweise
- Bei größeren Aufgaben erst einen Plan zeigen, dann umsetzen.
- Annahmen ausdrücklich nennen. Bei Unklarheit fragen.
- Nie „fertig“ melden, ohne zu zeigen, dass es funktioniert.
- Vor dem Löschen oder Überschreiben fragen.

## Über mich
- Profil: 00 Kontext/Über mich.md
```

Sag Claude

Lass Claude deine Hausordnung schreiben

> Leg mir eine globale CLAUDE.md an. Frag mich vorher nacheinander: In welcher Sprache und wie ausführlich soll ich antworten? Was soll ich vor größeren Aufgaben tun? Was darf ich nie ohne Rückfrage tun? Woran erkenne ich, dass eine Aufgabe wirklich fertig ist? Halte die Datei unter 40 Zeilen und verweise auf „00 Kontext/Über mich.md“.

Was passiert	Claude stellt vier Fragen und schreibt die Datei an den richtigen Ort.
Testen	Eine neue Sitzung starten: Claude hält sich an die Regeln, ohne dass du sie nennst. Beispiel Wochenbericht: Nach „Mach daraus eine Regel für dieses Projekt“ legt Claude eine CLAUDE.md im Vault an.

Fertig, wenn …

- [ ] auf „Welche CLAUDE.md-Dateien hast du geladen?“ nennt Claude deine globale Datei (im Terminal: `/memory`)
- [ ] in einer neuen Sitzung antwortet Claude im gewünschten Stil, ohne Hinweis von dir
- [ ] nach einer Korrektur hast du einmal „Mach daraus eine Regel“ gesagt

Stolpersteine

- ! Die Datei wird zu lang und Regeln gehen unter: lieber kürzen als ergänzen
- ! Globale und Projekt-Datei widersprechen sich: eine Stelle entscheiden lassen
- ! Passwörter oder Schlüssel gehören nie in die CLAUDE.md (Teil 5)

Weiter mit Teil 04: **Das Kurzzeitgedächtnis**

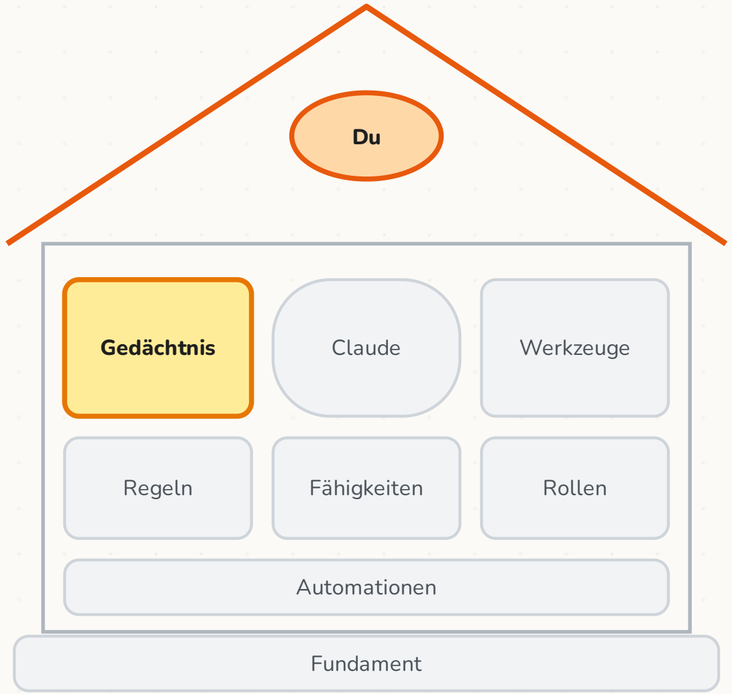
Zur Übersicht aller Teile

Teil 04 von 12 · Stufe A: Loslegen

Das Kurzzeitgedächtnis

Jede Sitzung beginnt dort, wo die letzte aufgehört hat.

ca. 30 Min. zum Mitmachen | Voraussetzung: Teil 3 | kostenlos



Drei Orte statt Zettelwirtschaft

Das Problem

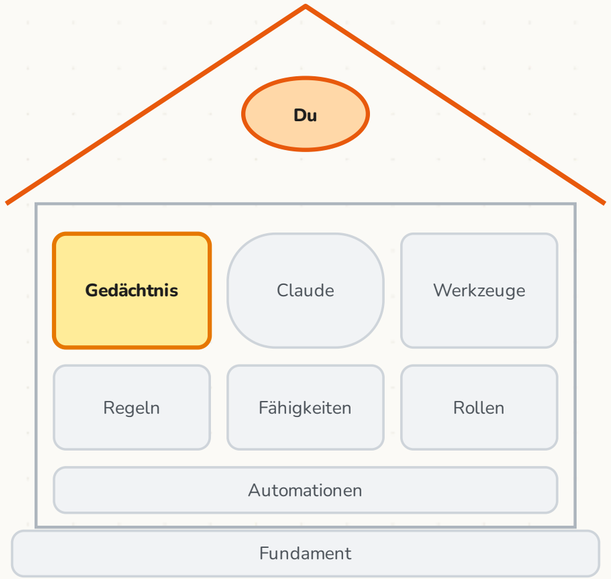
Claude vergisst zwischen zwei Sitzungen, was gestern lief. Und deine Einfälle landen irgendwo zwischen Handy, Mail und Kopf.

Das Prinzip

Ein Brain Dump für alles Unsortierte, **eine Tagesnotiz** je Tag, und an deren Ende **der Übertrag**: die einzige Liste für „was ist als Nächstes dran“.

So mache ich es

Ideen kommen per Sprachnachricht oder Tastatur in den Brain Dump. Bei „Feierabend“ sortiert Claude ihn ein, zum Start liest es den letzten Übertrag und prüft jeden Punkt, bevor er weiter gilt.



Zwei Routinen, jeden Tag gleich

„Guten Morgen"

- Tagesnotiz für heute anlegen
- Übertrag der letzten Notiz übernehmen, nichts doppelt
- Brain Dump lesen, Auffälliges melden
- Kurzbericht: Morgen zuerst, Entscheidungen, Wartendes

„Feierabend"

- Brain Dump einsortieren, Unklares nachfragen
- Übertrag aktualisieren, Erledigtes streichen
- Brain Dump erst nach deinem Ok leeren
- speichern (ab Teil 5: committen und hochladen)

Die Routinen stehen als Regeln in der CLAUDE.md. Ein Wort genügt, Claude kennt den Ablauf.

So sieht ein Übertrag aus

- → Feste Fächer, damit nichts zwischen den Stühlen liegt.
- → Eine Liste, kein Bericht. Was erledigt ist, fliegt raus.
- → Das Datum zeigt, was schon lange liegt.
- → „Bewusst nicht gemacht“ verhindert, dass dieselbe Idee jeden Tag neu auftaucht.

06 Tagesnotizen/2026-10-19.md (Ende)

```
## ⏭ Übertrag

### Morgen zuerst
- Angebot an Firma X fertig machen (seit 14.10.)

### Ich muss entscheiden
- Welches Hosting für die Website? (seit 12.10.)

### Wartet auf andere
- Rückmeldung Steuerberater (seit 10.10.)

### Bewusst nicht gemacht
- Newsletter: erst nach dem Start
```

Sag Claude

Lass Claude die Routinen einrichten

> Richte mir eine Tagesroutine ein: Leg „01 Inbox/Brain Dump.md“ an und für heute eine Tagesnotiz in „06 Tagesnotizen“ (Dateiname JJJJ-MM-TT.md), die mit dem Abschnitt „## ⏭ Übertrag“ endet, mit den Fächern Morgen zuerst, Ich muss entscheiden, Wartet auf andere, Bewusst nicht gemacht. Schreib die Start- und die End-Routine als Regeln in meine CLAUDE.md, sodass du sie ausführst, wenn ich „Guten Morgen“ oder „Feierabend“ sage.

Was passiert	Claude legt beide Dateien an und ergänzt deine Hausordnung um die zwei Routinen.
Testen	Morgen in einer neuen Sitzung „Guten Morgen“ schreiben.

Fertig, wenn …

- [ ] auf „Guten Morgen“ nennt Claude deinen Übertrag von gestern
- [ ] der Brain Dump ist nach dem Einsortieren leer
- [ ] es gibt genau einen Ort für „was ist als Nächstes dran“

Stolpersteine

- ! Der Übertrag wird zum Tagesbericht: Liste halten, Erledigtes streichen
- ! Zwei Listen für Offenes (Übertrag und Aufgaben-App): eine davon abschaffen
- ! Routine nur halb ausgeführt: Claude bitten, jeden Schritt sichtbar abzuhaken

Weiter mit Teil 05: **Sichern**

Zur Übersicht aller Teile

Teil 05 von 12 · Stufe B: Absichern und erweitern

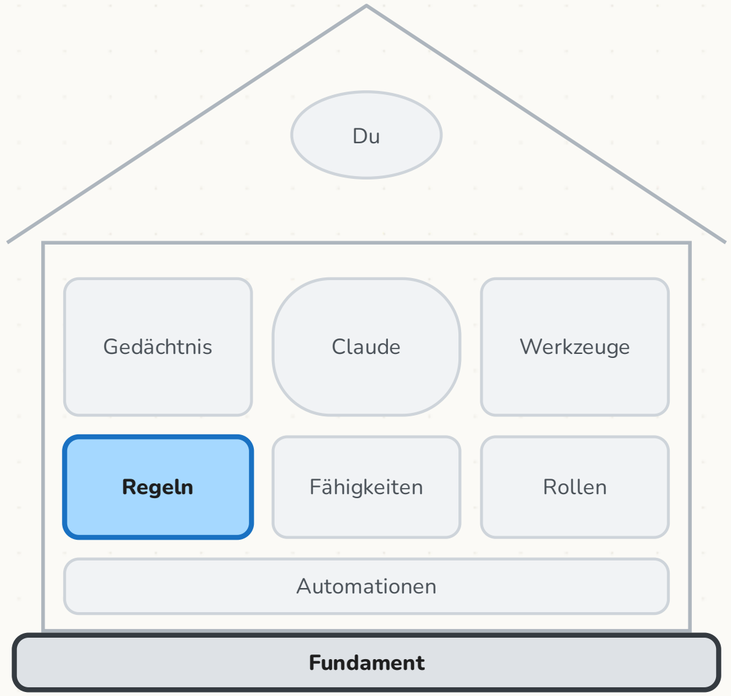
Sichern

Jede Änderung ist nachvollziehbar und liegt zusätzlich online. Schlüssel und Passwörter landen nie dort.

ca. 60 Min. zum Mitmachen

Voraussetzung: Teil 4

kostenlos



Ein Verlauf für alles, was Claude ändert

Das Problem

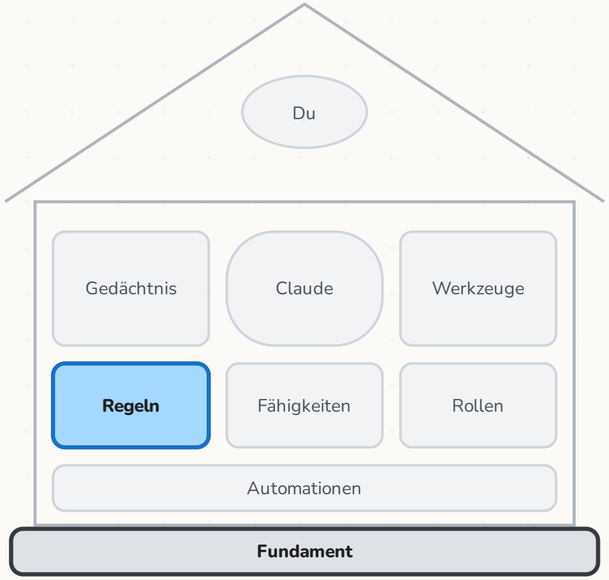
Claude ändert viele Dateien in kurzer Zeit. Ohne Verlauf lässt sich ein Fehler nicht zurückdrehen, und ein defekter Rechner nimmt alles mit.

Das Prinzip

Git merkt sich jeden Stand, GitHub hält eine Kopie online. Geheimnisse stehen in einer Datei `.env`, die nie hochgeladen wird, und gitleaks prüft vor jedem Hochladen.

So mache ich es

Mein ganzer Vault ist ein Git-Repo. Nach jedem abgeschlossenen Schritt wird gespeichert und sofort hochgeladen. Im April stand einmal ein Schlüssel im Repo; seitdem prüft gitleaks jeden Push.



Fünf Wörter, die du brauchst

Begriffe

- **Repo**: ein Ordner mit Verlauf
- **Commit**: ein Speicherpunkt mit kurzer Beschreibung
- **Push**: Speicherpunkte zu GitHub hochladen
- **Pull**: den neuesten Stand herunterladen
- **.gitignore**: Liste, was nie ins Repo darf

Regeln für Claude

- nach jedem abgeschlossenen Schritt committen und pushen
- Schlüssel nur in `.env`, nie in Notizen oder Code
- Repos privat anlegen
- die Prüfung vor dem Push nie umgehen
- vor dem Commit zeigen, was sich geändert hat

Was du brauchst

Git

Merkt sich jeden Stand deiner Dateien. Jede Änderung ist nachvollziehbar und rückgängig zu machen.

Kosten: kostenlos (Open Source)

Offizielle Seite ↗ GitHub ↗

GitHub

Online-Ablage für deine Repos: Sicherung und Abgleich zwischen Rechnern. Private Repos sind im kostenlosen Plan enthalten.

Kosten: kostenlos (Free-Plan)

Offizielle Seite ↗

GitHub CLI (gh)

GitHub im Terminal: anmelden, Repos anlegen. Claude benutzt es statt der Webseite.

Kosten: kostenlos

Offizielle Seite ↗ GitHub ↗

gitleaks

Prüft vor dem Hochladen, ob Passwörter oder Schlüssel in den Dateien stehen, und stoppt dann.

Kosten: kostenlos (MIT-Lizenz)

GitHub ↗ Anleitung ↗

Homebrew (nur Mac)

Paketverwaltung für den Mac. Damit installiert man Kommandozeilen-Programme mit einem Befehl.

Kosten: kostenlos

Offizielle Seite ↗ GitHub ↗

Auf dem Mac installiert Homebrew `gh` und `gitleaks`. Unter Windows übernimmt das `winget`, das schon dabei ist.

Werkzeuge installieren

1 Git

Mac · Terminal

```
xcode-select --install
```

Windows · PowerShell

```
winget install --id Git.Git -e --source winget
```

2 GitHub CLI und gitleaks

Auf dem Mac vorher Homebrew installieren (Befehl auf brew.sh). Unter Windows ist gitleaks ein Community-Paket; die offizielle Alternative ist die ZIP-Datei von der GitHub-Seite.

Mac · Terminal

```
brew install gh gitleaks
```

Windows · PowerShell

```
winget install --id GitHub.cli --source winget
winget install Gitleaks.Gitleaks
```

3 Bei GitHub anmelden

Neues Terminal-Fenster öffnen, dann im Browser bestätigen.

```
gh auth login
```

Sag Claude

Lass Claude das Repo einrichten

> Mach aus diesem Ordner ein privates Git-Repo und lade es mit gh auf GitHub hoch. Leg eine .gitignore an, die .env und alle Dateien mit Passwörtern ausschließt. Richte einen pre-push-Hook mit gitleaks ein, der das Hochladen stoppt, wenn ein Schlüssel gefunden wird. Ergänze meine CLAUDE.md um die Regeln: nach jedem abgeschlossenen Schritt committen und pushen, Schlüssel nur in .env, den Hook nie umgehen.

Was passiert Claude legt erst einen Plan vor und fragt nach Name und Inhalt. Nach deinem Go legt es Repo, .gitignore und Hook an, testet den Hook und lädt den ersten Stand hoch.

Prüfen Frag Claude: „Zeig mir die Speicherpunkte und ob das Repo privat ist.“ Oder sieh auf github.com nach: Das Repo ist als „Private“ markiert.

Fertig, wenn …

- [ ] das Repo liegt privat auf GitHub
- [ ] Claude zeigt dir deine Speicherpunkte mit Beschreibung (im Terminal: `git log`)
- [ ] beim Push meldet gitleaks „no leaks found“
- [ ] eine `.env` existiert und steht in der `.gitignore`

Stolpersteine

! Schlüssel nicht in den Chat kopieren, sondern selbst in die `.env` eintragen. Steht doch einer in einer Notiz, prüft Claude vor dem Commit und lädt nichts hoch; der Hook ist die zweite Sicherung

! „Befehl nicht gefunden“ nach winget: neues Terminal-Fenster öffnen

! Ein einmal hochgeladener Schlüssel bleibt im Verlauf: Schlüssel beim Anbieter sofort erneuern

Weiter mit Teil 06: **Werkzeuge anbinden**

Zur Übersicht aller Teile

Teil 06 von 12 · Stufe B: Absichern und erweitern

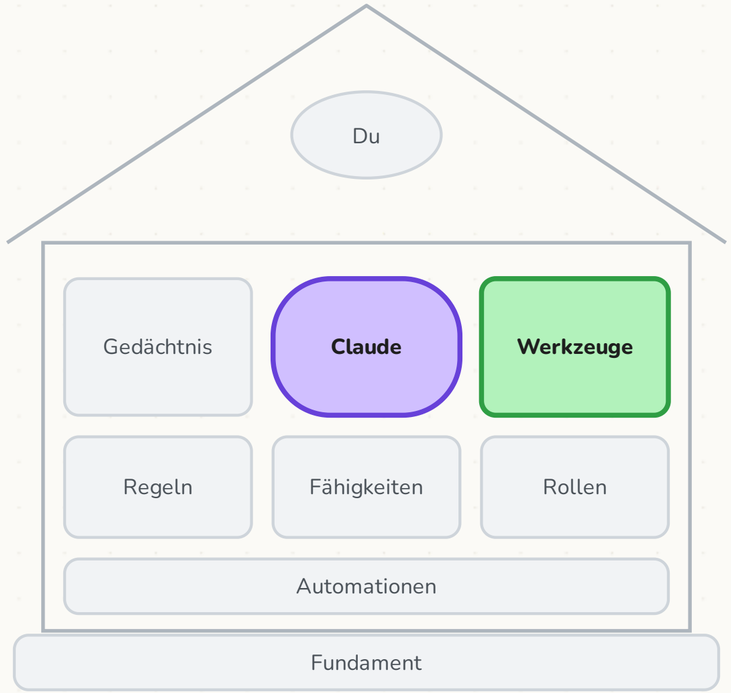
Werkzeuge anbinden

Claude kann mehr als Dateien: einen Browser bedienen, aktuelle Dokumentation lesen, Mail und Kalender sehen.

ca. 45 Min. zum Mitmachen

Voraussetzung: Teil 5

meist kostenlos



Hände für Claude

Das Problem

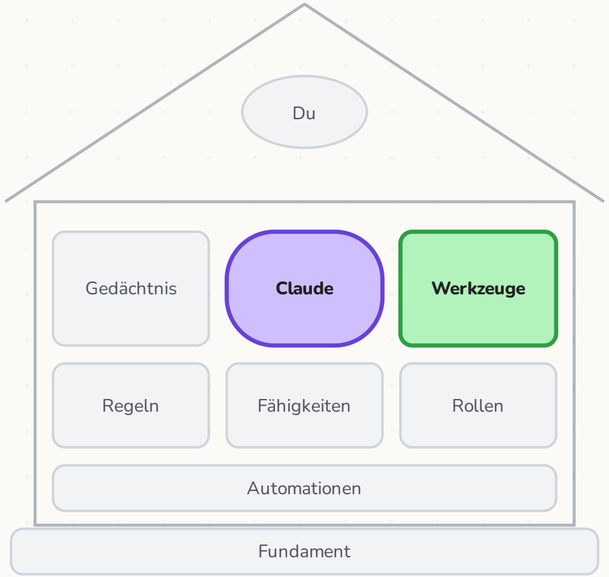
Claude kennt nur, was in seinem Ordner liegt und was es einmal gelernt hat. Aktuelle Webseiten, neue Software-Versionen und dein Kalender bleiben außen vor.

Das Prinzip

Zwei Arten von Werkzeugen. Kommandozeilen-Programme (CLI), die Claude wie du im Terminal aufruft. Und MCP-Server: kleine Adapter nach einem offenen Standard, die Claude die Werkzeuge eines Dienstes geben.

So mache ich es

Wo es ein gutes CLI gibt, nehme ich das, für GitHub zum Beispiel `gh`. Einen MCP-Server nur für das, was ein CLI nicht kann, etwa einen Browser fernsteuern.



CLI oder MCP?

CLI: ein Programm im Terminal

- Beispiele: `git`, `gh`, `vercel`
- Claude ruft es auf und liest die Ausgabe
- kostet nichts, solange es nicht benutzt wird
- viele kennt Claude schon

MCP: ein Adapter für einen Dienst

- Beispiele: Browser (Playwright), Doku (Context7), Web (Firecrawl)
- eingerichtet mit `claude mcp add …`
- Mail und Kalender als Connector per Klick in claude.ai
- `/mcp` zeigt, was verbunden ist

Erstes Werkzeug: dein Kalender

1 Connector verbinden

Auf **claude.ai** › „Anpassen“ › „Connectoren“ › **Google Calendar** › Verbinden, dann bei Google anmelden. Für Outlook gibt es den Connector **Microsoft 365**.

2 Nichts installieren

Claude Code übernimmt die Connectoren deines Claude-Kontos von selbst, auch in Claudian. Abschalten lassen sie sich mit `"disableClaudeAiConnectors": true` in den Einstellungen.

3 Ausprobieren

„Hol dir die Termine dieser Woche aus meinem Kalender und ergänze damit den Wochenbericht.“ Claude sucht das Werkzeug selbst, fragt den Kalender ab und schreibt die Termine hinein.

Drei Werkzeuge für den Anfang

Node.js (LTS)

Laufzeitumgebung. Sie liefert `npx`, mit dem die meisten MCP-Server starten. Claude Code selbst braucht sie nicht.

Kosten: kostenlos

Offizielle Seite ↗ GitHub ↗

Playwright MCP

Claude steuert einen echten Browser: Seiten öffnen, klicken, Screenshots, Oberflächen selbst testen.

Kosten: kostenlos

Offizielle Seite ↗ GitHub ↗

Context7 MCP

Holt die aktuelle Dokumentation einer Software, statt dass Claude aus altem Wissen rät.

Kosten: kostenloser Schlüssel für höhere Limits

Offizielle Seite ↗ GitHub ↗

Mail und Kalender (Connectoren)

Gmail und Google Kalender per Klick verbinden. Sie erscheinen automatisch in Claude Code, wenn dasselbe Konto angemeldet ist.

Kosten: in den Claude-Plänen enthalten

Anleitung ↗

Node.js zuerst, denn die MCP-Server starten über `npx`. Connectoren brauchen kein Node.

Weniger ist mehr: eine Messung

22 → 6

MCP-Server installiert, davon wirklich genutzt

50 → 2

Skills installiert, davon regelmäßig genutzt

664

ausgewertete Gespräche aus 60 Tagen

Jedes Werkzeug belegt bei jedem Start einen Teil von Claudes Aufmerksamkeit, auch wenn es nie gebraucht wird.
Meine Regel seitdem: Ein Werkzeug kommt dazu, wenn eine Aufgabe es verlangt. Was nie aufgerufen wird, wird abgeschaltet.

Sag Claude

Lass Claude die ersten Werkzeuge anschließen

> Ich möchte, dass du Webseiten im Browser testen und aktuelle Dokumentation nachschlagen kannst. Prüf, ob Node.js installiert ist, und hilf mir sonst bei der Installation. Richte dann die MCP-Server Playwright und Context7 ein. Sag mir, wann ich dich neu starten muss, und zeig mir danach mit /mcp, was verbunden ist.

Was passiert	Claude prüft die Voraussetzungen und trägt beide Server ein.
Testen	„Öffne die Seite meiner Firma und beschreib, was du siehst.“

Fertig, wenn …

- [ ] Claude schreibt die Termine der Woche aus deinem Kalender in den Bericht
- [ ] `/mcp` listet Playwright und Context7 als verbunden (im Terminal; in Claudian: frag Claude, welche Werkzeuge verbunden sind)
- [ ] Claude öffnet eine Webseite und beschreibt sie
- [ ] du kannst erklären, wann ein CLI reicht und wann ein MCP-Server nötig ist

Stolpersteine

- ! Ein neu eingetragener Server ist erst nach einem Neustart von Claude nutzbar
- ! Unter Windows braucht ein `npx`-Server manchmal den Umweg `cmd /c`; Claude kennt ihn
- ! Zu viele Server auf einmal: Claude wird langsamer und wählt schlechter

Weiter mit Teil 07: **Rezepte und Rollen**

Zur Übersicht aller Teile

Teil 07 von 12 · Stufe B: Absichern und erweitern

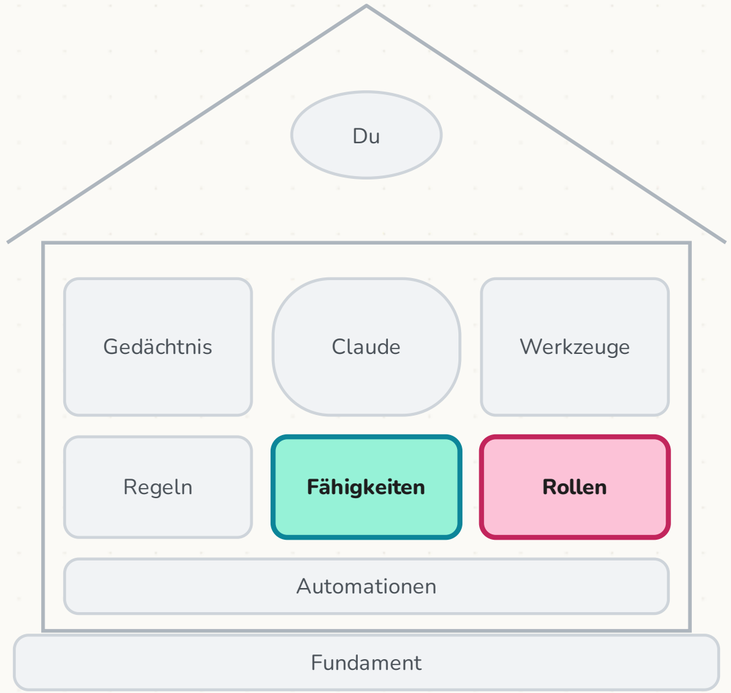
Rezepte und Rollen

Wiederkehrende Abläufe sind als Rezept gespeichert, und Claude gibt Teilarbeit an Helfer mit eigener Rolle ab.

ca. 45 Min. zum Mitmachen

Voraussetzung: Teil 3

kostenlos



Einmal erklären, immer wieder nutzen

Das Problem

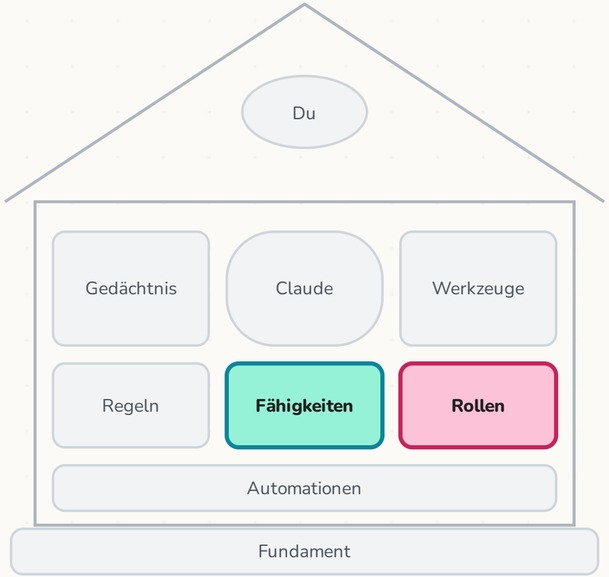
Gute Abläufe erklärst du immer wieder: wie ein Angebot aufgebaut ist, wie geprüft wird. Und eine einzige Sitzung, die alles selbst macht, verliert irgendwann den Überblick.

Das Prinzip

Ein Skill ist ein Rezept: ein Ordner mit einer Datei SKILL.md, die Claude lädt, sobald die Aufgabe passt. **Ein Agent ist eine Rolle**: ein weiterer Claude mit eigenem Auftrag und frischem Kopf.

So mache ich es

Elf Rollen, etwa Gegenleser, Tester und Sicherheitsprüfer. Claude verteilt die Arbeit und wählt je Auftrag das Modell: ein kleines für Suchen, ein mittleres für Standardarbeit, das stärkste für schwierige Entscheidungen.



Rezept oder Rolle?

Skill: wie etwas gemacht wird

- Ordner `~/.claude/skills/<name>/SKILL.md`
- die Beschreibung sagt, wann er greift
- Aufruf automatisch oder mit `/name`
- `/skills` listet alle
- Beispiel: grill-me, Angebot, Wochenbericht

Agent: wer es macht

- Datei `~/.claude/agents/<name>.md`
- eigener Kontext: liest nur, was er braucht
- arbeitet parallel, meldet ein Ergebnis zurück
- gut für Prüfen, Recherchieren, Gegenlesen
- Beispiel: Gegenleser, Tester

So einfach ist ein Rezept

- → Oben steht, wann der Skill greift: Claude liest nur diese Beschreibung, bis die Aufgabe passt.
- → Darunter die Schritte, so konkret wie bei einer Einarbeitung.
- → Vorlagen und Beispiele dürfen im selben Ordner liegen.
- → Ein Agent sieht genauso aus: Name, Beschreibung, Auftrag.

~/.claude/skills/angebot/SKILL.md

```
---
name: angebot
description: Erstellt ein Angebot nach
  meiner Vorlage. Auslösen bei „Angebot“,
  „Kostenvoranschlag“.
---

# Angebot
1. Kunde, Leistung, Preis erfragen,
   falls sie fehlen.
2. Vorlage „00 Kontext/Angebot.md“ nutzen.
3. Als PDF in „Meine Ablage“ speichern.
4. Entwurf zeigen, nichts versenden.
```

Zum Nachlesen und der erste fremde Skill

Skills

Rezepte: ein Ordner mit einer SKILL.md. Claude lädt ihn, sobald die Aufgabe zur Beschreibung passt.

Kosten: keine, eingebaut

Anleitung ↗

Subagenten

Rollen: weitere Claude-Instanzen mit eigenem Auftrag und frischem Kontext. Verbrauchen dasselbe Kontingent.

Kosten: Teil des Claude-Abos

Anleitung ↗

grill-me (Skill)

Claude fragt dich eine Frage nach der anderen, bis der Plan steht, bevor etwas gebaut wird.

Kosten: kostenlos

GitHub ↗ Anleitung ↗

Sag Claude

Lass Claude dein erstes Rezept schreiben

> Ich beschreibe dir einen Ablauf, den ich oft mache: [Ablauf in drei bis fünf Sätzen]. Mach daraus einen Skill in ~/.claude/skills/ mit einer Beschreibung, die sagt, wann du ihn nutzen sollst. Leg außerdem einen Agenten „gegenleser“ an, der Ergebnisse kritisch prüft, Fehler mit Fundstelle nennt und nichts selbst ändert.

Was passiert	Claude schreibt Skill und Agent und erklärt, wie beide ausgelöst werden.
Testen	Neue Sitzung, Auslöser nennen, etwa „Schreib den Wochenbericht für diese Woche“. In der Liste der Schritte steht dann „Skill wochenbericht“.

Fertig, wenn …

- [ ] dein Skill erscheint in der Liste der Schritte, wenn du den Auslöser nennst (im Terminal zeigt `/skills` alle)
- [ ] der Gegenleser hat ein Ergebnis einmal geprüft und etwas gefunden

Stolpersteine

- ! Eine vage Beschreibung („hilft bei Texten“) greift nie oder ständig
- ! Zu viele Skills: Claude wählt schlechter. Bei mir wurden 2 von 50 regelmäßig genutzt
- ! Ein Agent existiert erst, wenn seine Datei im Ordner `~/.claude/agents` liegt, nicht wenn er irgendwo beschrieben ist
- ! Eine neue Sitzung kennt frühere Gespräche nicht: Was dauerhaft gelten soll, gehört in die CLAUDE.md oder in den Skill

Weiter mit Teil 08: **Projekte planen**

Zur Übersicht aller Teile

Teil 08 von 12 · Stufe C: Verlässlich bauen

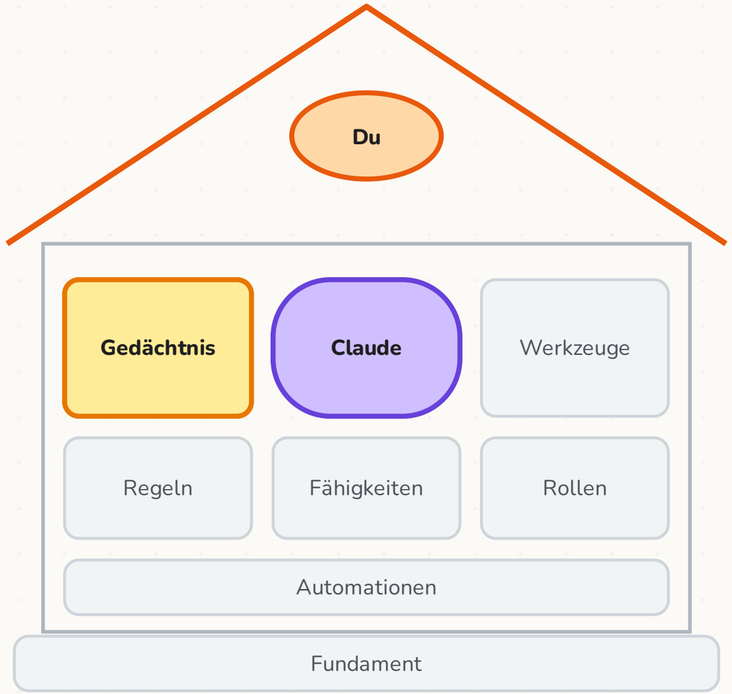
Projekte planen

Bevor gebaut wird, steht schriftlich fest, was gebaut wird und woran man merkt, dass es fertig ist.

ca. 30 Min. Mitmachen je Vorhaben

Voraussetzung: Teil 7

kostenlos



Erst klären, dann bauen

Das Problem

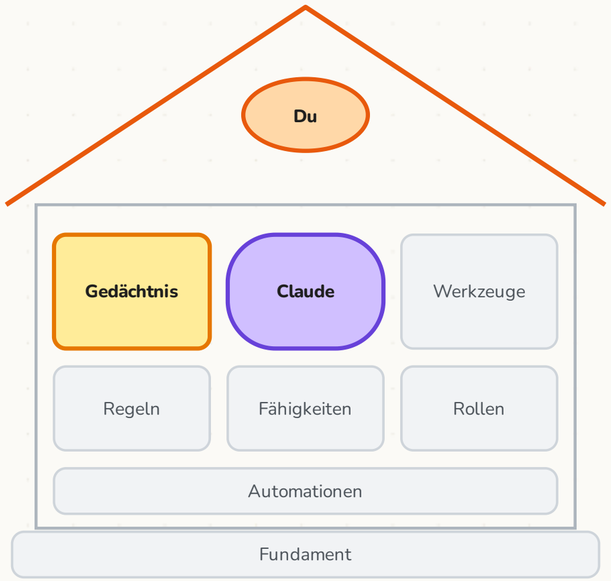
Ein unklarer Auftrag erzeugt schnell etwas, das niemand wollte. Und Beschlüsse von letzter Woche tauchen als neue Vorschläge wieder auf.

Das Prinzip

Erst ausfragen lassen, dann aufschreiben, dann bauen. Beschlüsse stehen mit Datum und Wortlaut in einer Datei und werden nicht neu verhandelt.

So mache ich es

Für jedes Vorhaben: `/grill-me`, bis jede Frage geklärt ist. Ergebnis ist ein KICKOFF (der Bauauftrag) und eine ENTSCHEIDUNGEN.md. Erst dann startet der Bau.



Von der Idee zum Bauauftrag

1. ## Idee in den Brain Dump

   Zwei Sätze reichen: was, für wen, warum jetzt.

2. ## Ausfragen lassen

   `/grill-me`: Claude stellt eine Frage nach der anderen, mit Optionen und Empfehlung, bis kein Ast offen ist.

3. ## KICKOFF.md schreiben

   Ziel, Umfang, was ausdrücklich nicht dazugehört, und Akzeptanzkriterien, die man prüfen kann.

4. ## ENTSCHEIDUNGEN.md anlegen

   Datum, Beschluss, dein Wortlaut. Was dort steht, schlägt Claude nicht neu vor.

5. ## In Etappen bauen

   Jede Etappe endet mit einem Prüfpunkt (Teil 9) und deiner Freigabe.

Prüfbar statt schwammig

- → „Die Seite ist schnell“ lässt sich nicht prüfen. „Lädt auf dem Handy in unter 2 Sekunden“ schon.
- → „Schönes Design“ wird zu: „Entwurf liegt als Link vor, ich habe ihn freigegeben“.
- → Jedes Kriterium nennt, wie es bewiesen wird: Befehl, Messwert, Screenshot.
- → Diese Anleitung ist genauso entstanden: Rechts ein Auszug ihrer Beschlussliste.

ENTSCHEIDUNGEN.md (Auszug, diese Anleitung)

```
# Entscheidungen

19.10.2026  Folien als HTML, teilbar per
            Link, später animierbar
19.10.2026  Grafiken handgezeichnet
19.10.2026  12 Teile in 4 Stufen
19.10.2026  Mac und Windows, kein Linux
19.10.2026  Jedes Werkzeug mit offiziellem
            Link oder GitHub-Repo
```

Sag Claude

Lass dich ausfragen

> Ich habe eine Idee: [zwei Sätze]. Nutze /grill-me und frag mich eine Frage nach der anderen, bis alles geklärt ist. Schreib danach eine KICKOFF.md mit Ziel, Umfang, Ausschlüssen und prüfbaren Akzeptanzkriterien und eine ENTSCHEIDUNGEN.md mit Datum und meinem Wortlaut. Bau noch nichts.

Was passiert Ein Interview von zehn bis zwanzig Fragen, danach zwei Dateien im Projektordner.

Ohne grill-me Der Prompt funktioniert auch ohne den Skill, dann ohne `/grill-me`.

Fertig, wenn …

- [ ] jedes Akzeptanzkriterium nennt seinen Nachweis
- [ ] du kannst in drei Sätzen sagen, was nicht gebaut wird
- [ ] die Beschlüsse stehen mit Datum in der ENTSCHEIDUNGEN.md

Stolpersteine

- ! Zu früh bauen, weil es so schnell geht: der Umbau kostet mehr als die Planung
- ! Kriterien wie „schön“, „schnell“, „einfach“ ohne Zahl oder Beleg
- ! Beschlüsse nur im Chat: nach der Sitzung sind sie weg

Weiter mit Teil 09: **Prüfen statt glauben**

Zur Übersicht aller Teile

Teil 09 von 12 · Stufe C: Verlässlich bauen

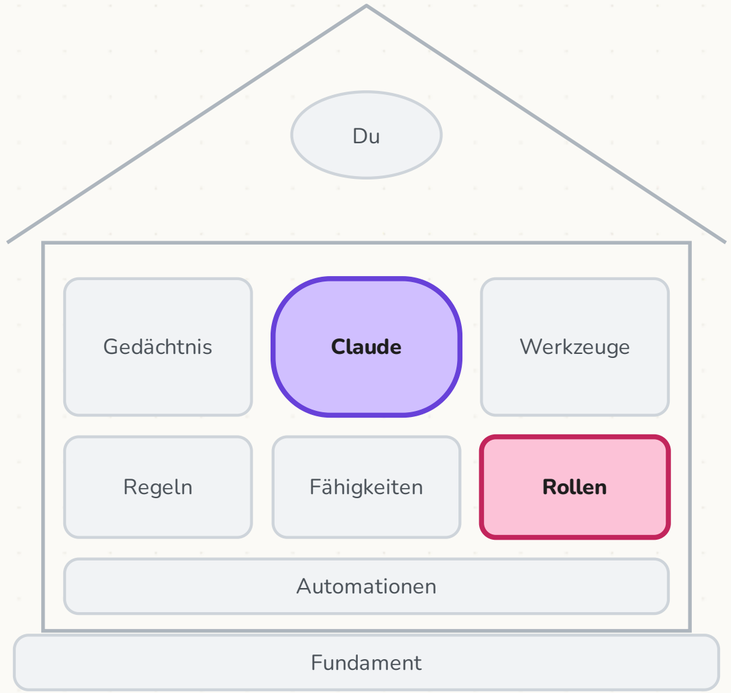
Prüfen statt glauben

„Fertig“ heißt bewiesen: Claude zeigt, dass es funktioniert, statt es zu behaupten.

ca. 45 Min. zum Mitmachen

Voraussetzung: Teil 8

kostenlos



Wer baut, prüft nicht allein

Das Problem

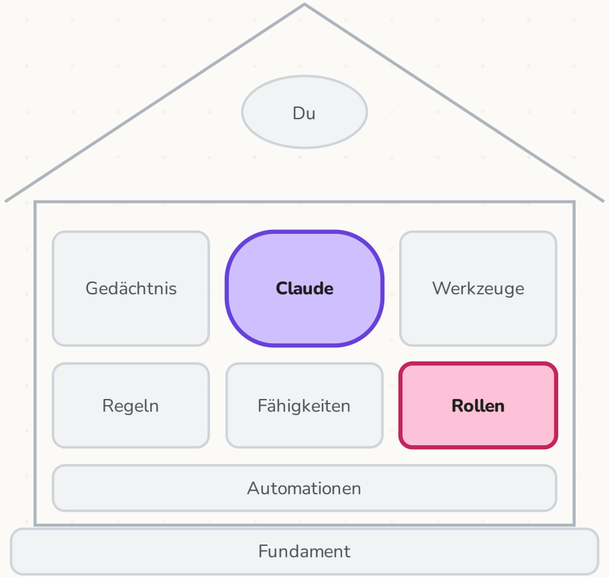
KI meldet gern „fertig“. Ob es wirklich läuft, ist eine andere Frage: Der Code kompiliert, aber der Knopf tut nichts.

Das Prinzip

Prüfen nach Risiko. Kleines wird ausgeführt und das Ergebnis gezeigt. Was andere sehen oder nutzen, prüfen unabhängige Prüfer, und keiner davon hat es gebaut.

So mache ich es

Für alles, was live geht, drei Prüfungen: Ein Gegenleser prüft den Inhalt, ein Tester das Verhalten, ein drittes Modell die Form. Erst wenn alle drei grün sind, ist eine Etappe fertig, und ich gebe sie frei.



Zwei Stufen nach Risiko

Alles andere

- Skripte, Notizen, Recherche, kleine Korrekturen
- ausführen und das Ergebnis zeigen
- Fehlschläge mit Ausgabe berichten
- kein Prüfprotokoll

Was live geht

- was Kunden sehen, Veröffentlichungen, Datenbanken
- Gegenleser: Ist es das Richtige?
- Tester: Funktioniert es, im Browser gesehen?
- Form: Sind alle Kriterien belegt?
- dann deine Freigabe

Eine Oberfläche gilt erst als geprüft, wenn jemand sie hat laufen sehen. „Kompiliert“ heißt nur: kompiliert.

Der eingebaute Prüfer: /goal

- → `/goal` setzt eine Bedingung. Claude arbeitet weiter, bis ein zweites, kleines Modell sie als erfüllt bestätigt.
- → Der Prüfer sieht nur das Gespräch, er führt selbst nichts aus. Die Bedingung muss deshalb sagen, wie es bewiesen wird.
- → Eine Obergrenze gehört hinein, damit es nicht endlos läuft.
- → `/goal` ohne Text zeigt den Stand, `/goal clear` bricht ab.
- → In Claudian läuft `/goal` auch, die Anzeige des Prüfers siehst du aber nur im Terminal (oder im Obsidian-Plugin „Terminal“, in VS Code).

Beispiel

```
/goal Alle Tests im Ordner tests/ laufen grün
(npm test endet mit Code 0), keine andere Datei
wurde geändert, oder Schluss nach 20 Runden.
```

Prüfwerkzeuge, die schon da sind

/goal (eingebaut)

Claude arbeitet weiter, bis ein zweites, kleines Modell eine Bedingung als erfüllt bestätigt.

Kosten: Teil des Claude-Abos

Anleitung ↗

/code-review und /security-review

Eingebaute Prüfbefehle: Fehlersuche im Code und Sicherheitsprüfung der Änderungen.

Kosten: Teil des Claude-Abos

Anleitung ↗

Playwright MCP

Claude steuert einen echten Browser: Seiten öffnen, klicken, Screenshots, Oberflächen selbst testen.

Kosten: kostenlos

Offizielle Seite ↗ GitHub ↗

Sag Claude

Mach Prüfen zur Regel

> Füge in meine CLAUDE.md eine Regel „Prüfen nach Risiko“ ein: Kleine Aufgaben führst du aus und zeigst das Ergebnis. Für alles, was andere sehen oder nutzen, lässt du vor „fertig“ einen Agenten „tester“ das Verhalten prüfen (Oberflächen im Browser) und den „gegenleser“ den Inhalt. Melde nie „fertig“ ohne Beleg: Befehl, Ausgabe oder Screenshot.

Was passiert	Claude ergänzt die Hausordnung und legt den Tester-Agenten an.
Testen	Bei der nächsten Aufgabe kommt „fertig“ mit Beleg.

Fertig, wenn …

- [ ] Claude nennt bei „fertig“ den Beleg
- [ ] du hast `/goal` einmal mit einer Bedingung samt Nachweis benutzt
- [ ] der Tester hat ein Ergebnis mit Befehl und Ausgabe belegt (bei Oberflächen: im Browser durchgeklickt)

Stolpersteine

- ! Ein Prüfer, der nur die Form prüft (gibt es die Datei?), ist eine Fassade
- ! Derselbe Agent baut und prüft: Er übersieht dieselben Fehler zweimal
- ! `/goal` ohne Obergrenze kann lange laufen und Kontingent verbrauchen

Weiter mit Teil 10: **Die Knowledge Base**

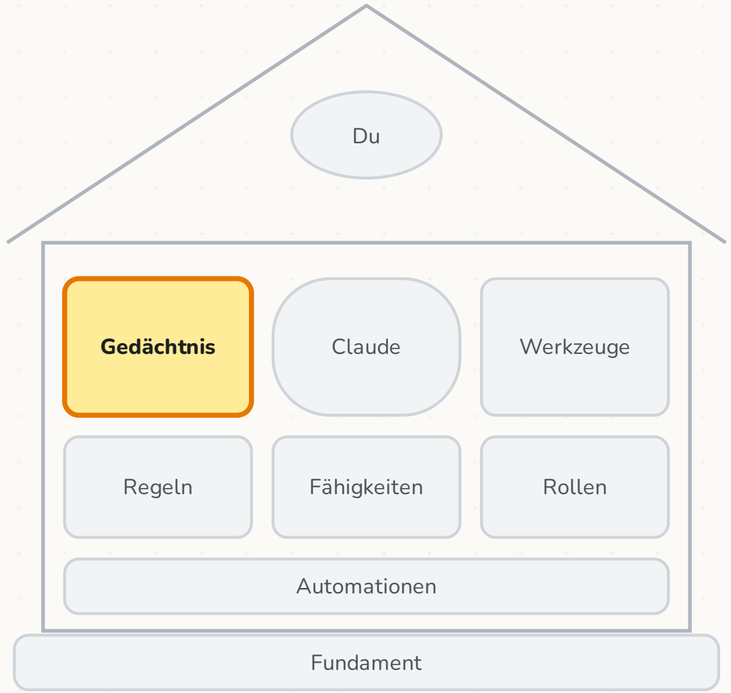
Zur Übersicht aller Teile

Teil 10 von 12 · Stufe D: Das System lernt und läuft allein

Die Knowledge Base

Was du einmal gelernt hast, findet Claude wieder, in jedem Projekt.

ca. 45 Min. zum Mitmachen | Voraussetzung: Teil 5 | kostenlos



Lernen, das bleibt

Das Problem

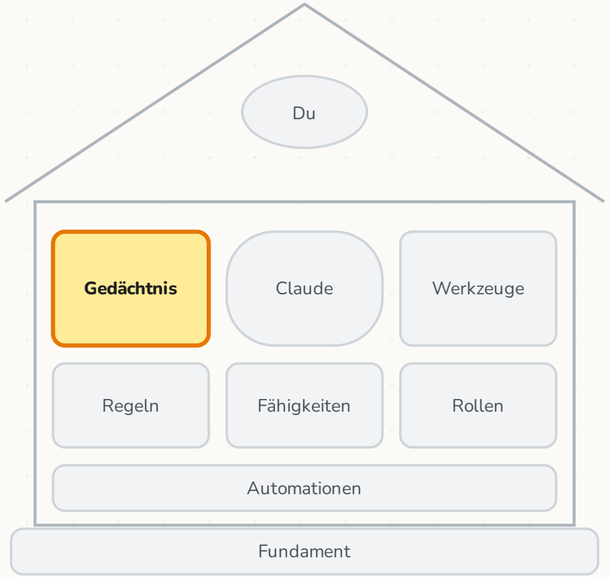
Gelerntes steckt in alten Gesprächen und Projektordnern. Im nächsten Projekt passiert derselbe Fehler noch einmal.

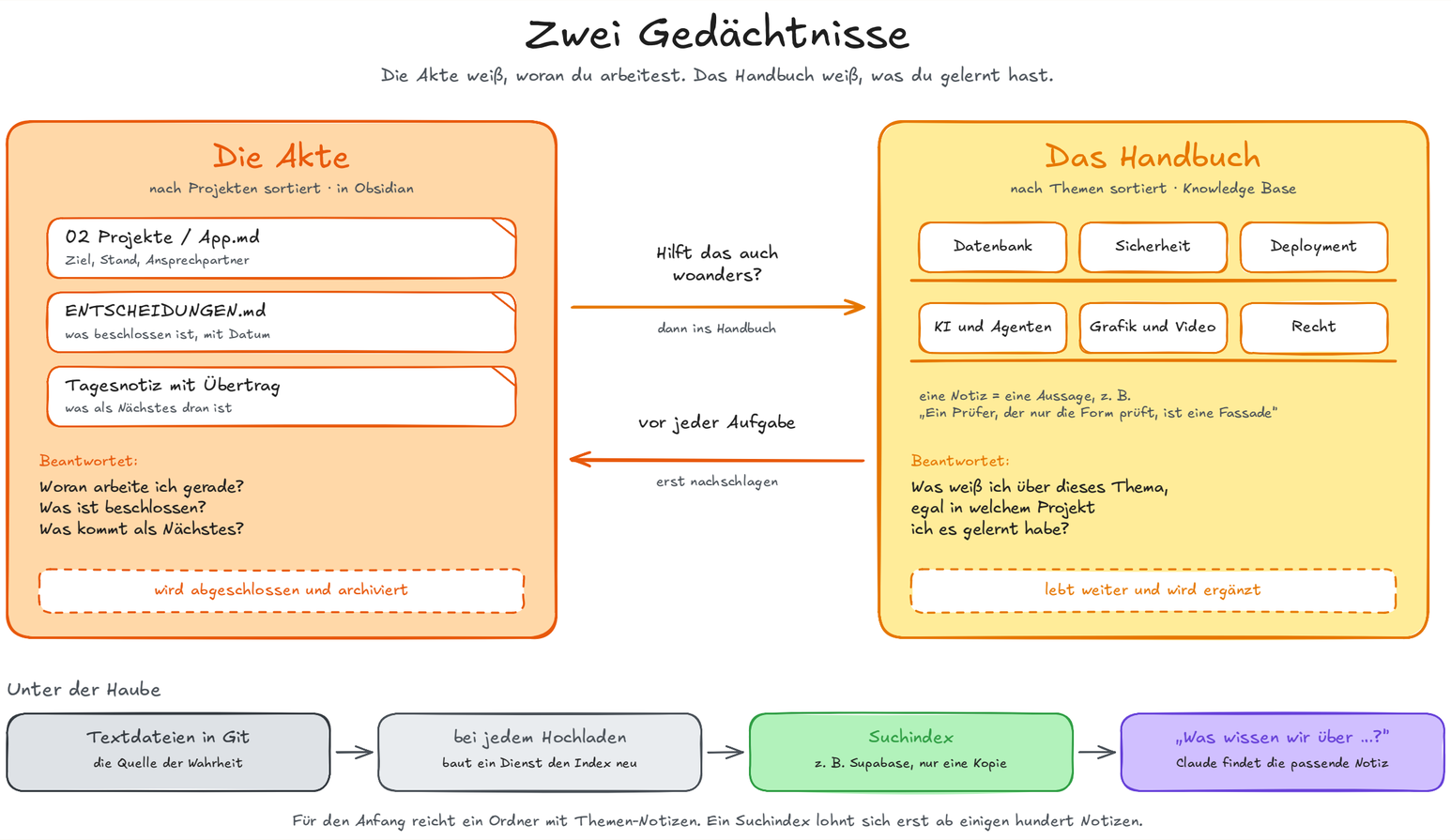
Das Prinzip

Zwei Gedächtnisse. Die Akte ist nach Projekten sortiert, das Handbuch nach Themen. Jede Lektion wird eine kurze Notiz, deren Titel die Erkenntnis ist.

So mache ich es

Rund 1.200 Notizen in vier Bereichen und 23 Themen. Vor jeder größeren Aufgabe sucht Claude darin, nach jeder Etappe kommt Neues dazu.





Akte und Handbuch

Die **Akte** beantwortet: Woran arbeite ich, was ist beschlossen, was kommt als Nächstes? Sie wird abgeschlossen und archiviert.

Das **Handbuch** beantwortet: Was weiß ich über dieses Thema, egal aus welchem Projekt? Es lebt weiter.

Die Prüffrage beim Ablegen: **Hilft das auch woanders?**

Anatomie einer Notiz

- → **Titel = die Aussage.** Wer nur die Titel liest, kennt schon die Lektionen.
- → **laden_wenn** sagt Claude, in welcher Lage die Notiz wichtig ist.
- → **Kontext** in ein bis vier Sätzen: Was ist passiert?
- → Vor dem Anlegen suchen: Gibt es die Aussage schon, wird ergänzt statt verdoppelt.
- → Kundendaten nie roh, nur anonymisiert.

05 Knowledge Base/prüfen/Ein Prüfer, der nur ...md

```
---
title: Ein Prüfer, der nur die Form prüft,
  ist eine Fassade
thema: pruefen
typ: lektion
laden_wenn: Prüfschritte planen
---

## Kontext
Was ist passiert? (1 bis 4 Sätze)

## Erkenntnis
Die eine Aussage, fett.

## Details
Befehle, Zahlen, Belege.
```

Klein anfangen, später suchen lassen

Am Anfang

- ein Ordner `05 Knowledge Base` mit Themen-Unterordnern
- Claude findet Notizen über Dateinamen und Volltextsuche
- neue Notizen am Ende jeder Etappe, nach jeder Korrektur, zum Feierabend

Ab einigen hundert Notizen

- ein Suchindex, der auch nach Bedeutung sucht, nicht nur nach Wörtern
- bei mir: Supabase, neu gebaut bei jedem Hochladen
- die Notizen bleiben Textdateien in Git; der Index ist nur eine Kopie

Fällt der Index weg, ist nichts verloren. Die Quelle der Wahrheit sind die Dateien.

Sag Claude

Lass Claude dein Handbuch anlegen

> Leg einen Ordner „05 Knowledge Base“ an, mit Unterordnern für vier bis sechs Themen, die zu meiner Arbeit passen (frag mich danach). Schreib eine Vorlage für eine Notiz: Titel ist die Aussage, dann laden_wenn, Kontext, Erkenntnis, Details. Ergänze meine CLAUDE.md: Vor jeder größeren Aufgabe suchst du dort, und am Ende jeder Etappe schlägst du mir neue Notizen vor. Projektstände gehören nicht hinein, nur was auch woanders hilft.

Was passiert Claude fragt nach deinen Themen, legt Ordner und Vorlage an und ergänzt die Hausordnung. Dann erzählst du ihm einen Fehler, und es macht daraus die erste Notiz.

Testen Neue Sitzung, dieselbe Aufgabe wie beim Fehler: Claude liest die Notiz zuerst und macht ihn nicht noch einmal. Oder frag: „Was wissen wir über …?“

Fertig, wenn …

- [ ] die ersten drei Notizen existieren, jede mit einer Aussage im Titel
- [ ] auf „Was wissen wir über …?“ findet Claude die passende Notiz
- [ ] deine Hausordnung verlangt Nachschlagen vor größeren Aufgaben

Stolpersteine

- ! Projektstände im Handbuch: die gehören in die Akte
- ! Mehrere Aussagen in einer Notiz: lieber zwei Notizen
- ! Notizen nie ergänzen, nur neu anlegen: Dubletten widersprechen sich irgendwann

Weiter mit Teil 11: **Automationen**

Zur Übersicht aller Teile

Teil 11 von 12 · Stufe D: Das System lernt und läuft allein

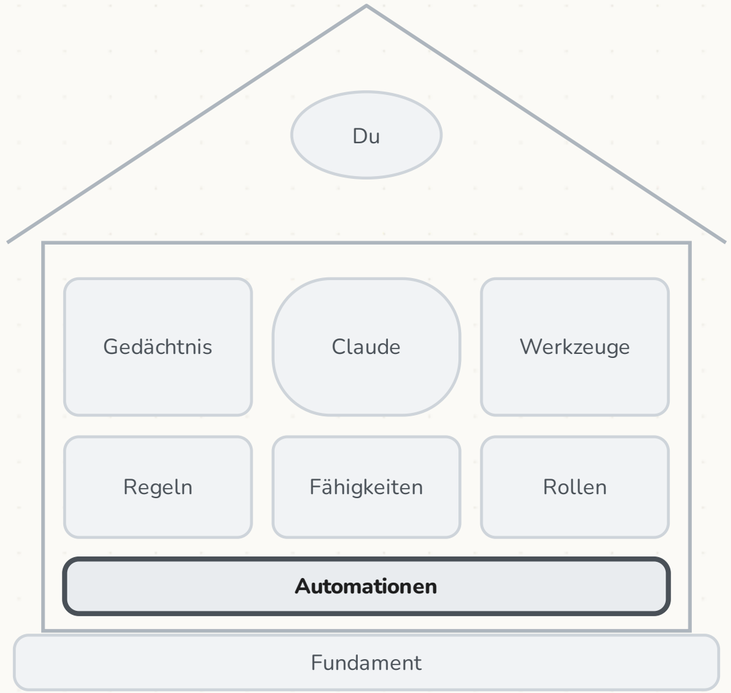
Automationen

Wiederkehrendes läuft von selbst: beim Start, beim Ende, nach Zeitplan oder wenn etwas passiert.

ca. 60 Min. zum Mitmachen

Voraussetzung: Teil 5

kostenlos bis wenige Euro



Haustechnik

Das Problem

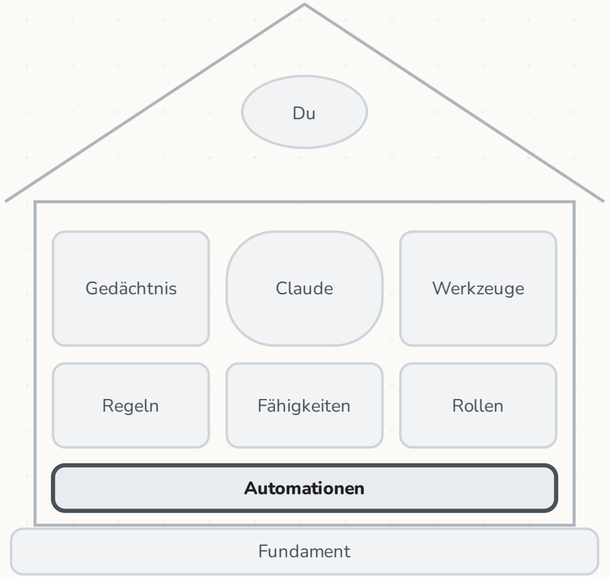
Routinen vergisst man genau dann, wenn es stressig wird: hochladen, sichern, aufräumen, nachsehen.

Das Prinzip

Drei Auslöser. Hooks reagieren auf Ereignisse in Claude Code. Zeitpläne starten Aufgaben zu festen Zeiten. Ein Workflow-Werkzeug verbindet Dienste: wenn X passiert, dann Y.

So mache ich es

Beim Start holt ein Hook den neuesten Stand, beim Ende speichert er Wissen und Tagesnotiz und lädt hoch. Nachts läuft ein verschlüsseltes Backup. Sprachnotizen vom Handy landen von selbst im Brain Dump.



Welcher Auslöser wofür?

In Claude Code

- **Hooks**: beim Start, beim Ende, vor oder nach einem Werkzeug
- `/hooks` zeigt, was eingerichtet ist
- `/loop`: eine Aufgabe in der laufenden Sitzung wiederholen
- `/schedule`: Routinen, die in der Cloud laufen

Außerhalb

- Zeitpläne des Betriebssystems (Mac: launchd, Windows: Aufgabenplanung)
- **n8n**: Dienste verbinden, etwa Telegram → Notiz
- GitHub Actions: etwas tun, sobald hochgeladen wird
- jede Automation meldet Fehler, statt still zu scheitern

Ein Hook in sechs Zeilen

- Hooks stehen in `~/.claude/settings.json` (global) oder `.claude/settings.json` (Projekt).
- Das Beispiel holt beim Start jeder Sitzung den neuesten Stand von GitHub.
- Ein Hook läuft mit deinen Rechten: nur Befehle eintragen, die du verstehst.
- Claude schreibt Hooks gern selbst, und zeigt sie vorher.

~/.claude/settings.json (Ausschnitt)

```
{
  "hooks": {
    "SessionStart": [
      { "hooks": [
        { "type": "command",
          "command": "git pull --ff-only" }
      ] }
    ]
  }
}
```

Freitags von allein: eine geplante Aufgabe

1 Claude fragen

„Wie lasse ich den Wochenbericht jeden Freitag um 14 Uhr von allein schreiben? Erklär es mir kurz, ändere noch nichts.“ Claude nennt die Wege. Frag nach, wenn etwas nicht passt: In der Aufnahme hat Claude einen Fehler erst auf Nachfrage korrigiert.

2 Ohne Terminal: Claude-App

Code › **Scheduled** › **New task**, Name und Anweisung eintragen, unter Advanced settings **Only on this computer** und den Vault-Ordner wählen, Zeitplan Freitag 14:00. Die App muss offen und der Rechner wach sein. Anleitung: code.claude.com/docs/en/desktop-scheduled-tasks

3 Ohne eigenen Rechner: Cloud

`/schedule` legt eine Routine in der Cloud an. Sie arbeitet auf einer frischen Kopie deines GitHub-Repos und sieht nur, was dort liegt.

Werkzeuge für Automationen

Hooks

Befehle, die Claude Code automatisch ausführt, etwa beim Start oder Ende einer Sitzung.

Kosten: keine, eingebaut

Anleitung ↗

Zeitpläne in Claude

`/loop` wiederholt eine Aufgabe in der laufenden Sitzung, `/schedule` legt Routinen an, die in der Cloud laufen.

Kosten: Teil des Claude-Abos

Anleitung ↗

n8n

Automationen per Klick-Editor: wenn X passiert, dann Y, auch mit KI-Schritten.

Kosten: Cloud ab 20 EUR/Monat; selbst betrieben kostenlos, aber laut Lizenz nur für interne oder nicht-kommerzielle Zwecke

Offizielle Seite ↗ GitHub ↗ Anleitung ↗

Telegram-Bot

Eingang und Alarmkanal: Sprachnotizen unterwegs ans System schicken, Meldungen aufs Handy bekommen.

Kosten: kostenlos

Offizielle Seite ↗ Anleitung ↗

Sag Claude

Lass Claude den ersten Hook bauen

> Richte mir einen Hook ein: Wenn eine Sitzung endet, sollen Tagesnotiz und Knowledge Base committet und gepusht werden. Fehler sollen in eine Log-Datei geschrieben werden, damit nichts still scheitert. Zeig mir vorher, was du in die settings.json schreibst, und teste es danach mit einer kurzen Sitzung.

Was passiert Claude zeigt dir erst die settings.json und den Testplan, schreibt dann ein kleines Skript, trägt es als SessionEnd-Hook ein und testet es mit einer kurzen Sitzung.

Prüfen Nach dem Sitzungsende liegt ein neuer Commit auf GitHub, und das Log meldet „gepusht“.

Fertig, wenn …

- [ ] `/hooks` zeigt deinen Hook
- [ ] nach einer Sitzung liegt ein neuer Commit auf GitHub, ohne dass du etwas getan hast
- [ ] ein absichtlich ausgelöster Fehler steht in der Log-Datei

Stolpersteine

- ! Eine Automation, die lautlos nicht auslöst, ist die gefährlichste Sorte: immer einen Beleg einbauen
- ! n8n selbst betrieben ist laut Lizenz nur für interne oder nicht-kommerzielle Zwecke frei
- ! Hooks laufen ohne Rückfrage: keine Befehle, die etwas löschen

Weiter mit Teil 12: **Mehrere Rechner und Dauerbetrieb**

Zur Übersicht aller Teile

Teil 12 von 12 · Stufe D: Das System lernt und läuft allein

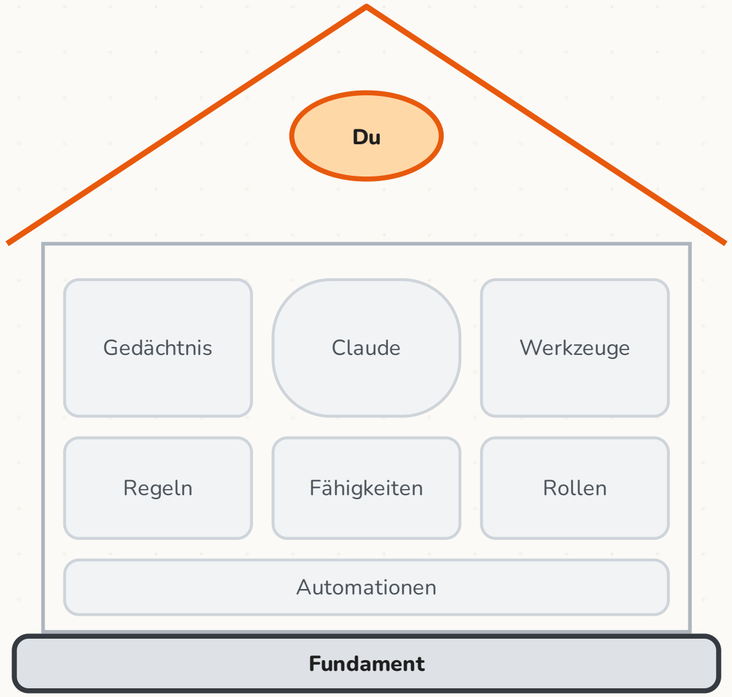
Mehrere Rechner und Dauerbetrieb

Dein System läuft auf mehr als einem Gerät, und manches läuft rund um die Uhr, auch wenn dein Rechner aus ist.

nach Bedarf

Voraussetzung: Teile 5, 10, 11

ab wenigen Euro im Monat



Mehr als ein Rechner

Das Problem

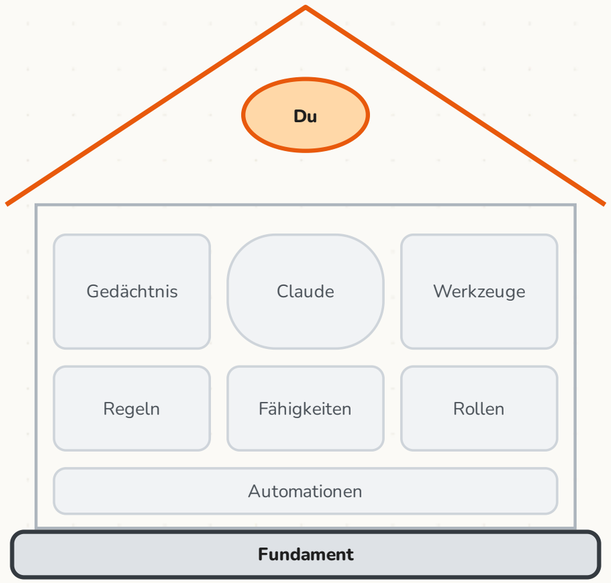
Ein Rechner ist ein einzelner Ausfallpunkt. Und manche Aufgaben brauchen jemanden, der wach ist, wenn du schläfst.

Das Prinzip

Git ist die Brücke zwischen Geräten. Jedes Gerät hat eine Rolle. Ein Server übernimmt den Dauerbetrieb, mit so wenig Rechten wie nötig.

So mache ich es

Ein Rechner zum Entwickeln, einer fürs Geschäft, ein Server für Automationen und Wächter. Ein Arbeitsstrang wechselt per Übergabe-Notiz das Gerät. Ein zweiter Assistent darf Entwürfe schreiben, aber nichts versenden.



Übergabe und Dauerbetrieb

Von Gerät zu Gerät

- erst holen (pull), dann arbeiten, dann hochladen (push)
- große Dateien in einen Speicher, nicht in Git
- eine Übergabe-Notiz: Stand, nächster Schritt, offene Fragen, Belege
- auf dem anderen Gerät: „Mach da weiter, wo ich aufgehört habe“

Rund um die Uhr

- ein kleiner Server für Automationen und Wächter
- Wächter melden per Telegram, statt still zu scheitern
- Backups verschlüsselt, Wiederherstellung einmal geprobt
- Assistenten mit Grenzen: lesen und entwerfen ja, senden nur nach Freigabe

Im Oktober hatte mein Entwicklungsrechner ein Startproblem. Danach entstanden ein Notfallplan und eine Probe-Wiederherstellung.

Eine Übergabe-Notiz

- → Jede Aussage hat einen Beleg: Commit, Datei, Messwert.
- → Was nicht belegt ist, steht ausdrücklich als „ungeprüft“ da.
- → Die Notiz liegt im Repo; das andere Gerät findet sie beim Start.
- → Ist der Strang abgeschlossen, wird die Notiz gelöscht.

KONTEXT-website.md

```
# Übergabe: Website

Stand: Startseite und Kontaktformular
  fertig (Commit a1b2c3d, Test grün)
Nächster Schritt: Impressum, Texte in
  02 Projekte/Website.md
Offen: Hosting, siehe ENTSCHEIDUNGEN.md
Ungeprüft: Darstellung auf dem iPhone
```

Was dafür hilft

Telegram-Bot

Eingang und Alarmkanal: Sprachnotizen unterwegs ans System schicken, Meldungen aufs Handy bekommen.

Kosten: kostenlos

Offizielle Seite ↗ Anleitung ↗

n8n

Automationen per Klick-Editor: wenn X passiert, dann Y, auch mit KI-Schritten.

Kosten: Cloud ab 20 EUR/Monat; selbst betrieben kostenlos, aber laut Lizenz nur für interne oder nicht-kommerzielle Zwecke

Offizielle Seite ↗ GitHub ↗

Anleitung ↗

GitHub

Online-Ablage für deine Repos: Sicherung und Abgleich zwischen Rechnern. Private Repos sind im kostenlosen Plan enthalten.

Kosten: kostenlos (Free-Plan)

Offizielle Seite ↗

Einen Server zu mieten ist eine eigene Entscheidung (Anbieter, Betrieb, Kosten). Claude hilft beim Vergleich und beim Einrichten.

Sag Claude

Lass Claude den Umzug vorbereiten

> Ich will mein System auf einem zweiten Rechner nutzen ([Mac oder Windows]). Prüf, ob alles Nötige in Git liegt und was nur auf diesem Rechner existiert. Schreib mir eine Anleitung „Neuer Rechner“ mit allen Installationsschritten und eine Vorlage für Übergabe-Notizen: Stand, nächster Schritt, offene Fragen, Belege.

Was passiert	Claude findet, was fehlt (Schlüssel, große Dateien, Einstellungen), und schreibt beides.
Prüfen	Auf dem zweiten Rechner kennt Claude nach dem Klonen deine Regeln und den Übertrag.

Fertig, wenn …

- [ ] auf dem zweiten Gerät startet Claude mit deinen Regeln und deinem Übertrag
- [ ] eine Übergabe-Notiz hat einen Strang sauber übergeben
- [ ] ein Backup wurde einmal testweise wiederhergestellt

Stolpersteine

- ! Zwei Geräte ändern gleichzeitig dieselbe Datei: immer erst holen, dann arbeiten
- ! Große Dateien in Git machen jedes Repo langsam
- ! Ein Server mit allen Rechten: Wächter brauchen nur Lesen und Melden

Ab hier: Frag Claude. Jede Lücke ist eine gute Frage.

Zur Übersicht aller Teile